



IBM BOB 2.0 HACKATHON · DEBUGGING & TESTING

Paste a stack trace. Get a **proven** fix.

Production traceback in. Verified failing test, suspect commit and verified fix out. IBM Bob writes the code. TraceFix proves it.

THE PROBLEM

3 a.m. Checkout is throwing 500s. All you have is a stack trace.

STEP 1

Find the code

Map log paths like
`/srv/app/...` to files in
the repo.

STEP 2

Find the change

Which of this week's
commits broke it?

STEP 3 · OFTEN SKIPPED

Reproduce it

Write a test that fails
the same way
production did.

STEP 4

Fix it safely

Without breaking
anything else, at 3 a.m.

Skipping the reproduction is why the same bugs come back. And "just ask an AI" is risky: who checks that its fix is actually right?

THE SOLUTION

One command. Five verified outputs.

```
$ tracefix run prod_trace.txt --repo ./shopcart --id INC-4127
```

- 01 Incident brief: every frame mapped to a repo file, with code context
- 02 Suspect commit: `git blame` around the crash lines, ranked
- 03 A regression test `proven` to fail exactly like production
- 04 A fix `proven` to pass that test and keep the whole suite green
- 05 An incident branch plus a report ready to use as the PR description

HOW IT WORKS

Bob proposes. TraceFix proves.

DETERMINISTIC

Parse & map

Noisy logs,
chained
exceptions, 3.11
carets.

DETERMINISTIC

Blame

Weighted
ranking of the
commits that
touched the
crash lines.

IBM BOB

Reproduce

Writes
`test_tracefix_repro.py`
and runs it.

IBM BOB

Fix

Minimal diff
that keeps the
intent of the
suspect
commit.

DETERMINISTIC

Ship

Test commit, fix
commit, report.

Repro gate

Same exception type · raised from the same function as
production · no other file touched.

Fix gate

Repro test now passes · full existing suite green · repro
test untouched.

Rejected? The exact pytest evidence goes back to the same Bob session. Up to 3 attempts.

From stack trace to proven fix in under a minute.

INC-4127 · ZERODIVISIONERROR

36s

A cart with only free promo items crashed the loyalty-points maths added in a *perf* commit. Bob's fix guards the edge case and keeps the optimisation.

INC-4133 · KEYERROR: ''

47s

A trailing comma from the mobile app ("SAVE10,") broke stacked coupons. Two Bob agents raced in parallel; bob#2 won.

Suspect found

The regression commit ranked #1 in both incidents.

Feedback loop

A rejected first attempt was corrected automatically on retry.

Tests stay behind

Every incident adds a regression test to the suite.

PARALLEL AGENTS

--agents N races Bob in isolated git worktrees.

```
Bob · reproduce    bob#1 + bob#2 · separate worktrees, different strategies
| bob#2 > Writing file tests/test_tracefix_repro.py
✓ VERIFIED bob#2 attempt 1: KeyError raised from _stacked_rate() – matches production
■ bob#1 stopped – another agent already won      (ACP session/cancel)
Bob · fix          ok in 19.2s (1 attempt(s), winner bob#2 of 2)
```

The first verified patch wins and is applied to the incident branch. Losing sessions are cancelled. More agents means more chances of a quick, correct answer, and the gates mean speed never costs correctness.

HOW WE USED IBM BOB

Bob built TraceFix, and Bob runs inside it.

BOB AS BUILDER · AGENT MODE

SESSION	BUILT
01	Demo service, git history & production trace (checked by re-running the crash)
02	Trace parser, repo mapper, blame ranking + 38 tests
03	47 tests for gates, reports, prompts

BOB AS ENGINE · RUNTIME

TraceFix drives Bob Shell over the **Agent Client Protocol** (`bob acp`):

- full repository context in each session
- Bob explores, writes, runs pytest itself
- parallel sessions + `session/cancel`
- reuses SSO login, no API keys

Every Bob build session's full event log is committed in `bob_sessions/`.

BUSINESS VALUE

Shorter incidents. Fewer repeat bugs. AI changes you can trust.

Shorter incidents

Getting from a stack trace to a reproduction and a fix takes seconds instead of a long night of digging.

Fewer repeat bugs

Each incident leaves behind a test that reproduces the real production failure, not a guessed one.

Safe adoption

Deterministic gates make AI-written changes reviewable and auditable, which makes them easier to get approved.

NEXT

Sentry / Datadog webhook → automatic incident branch

auto-open PR with report

Java, Node & Go traces

Bob custom mode for on-call



Bob proposes. TraceFix proves.

Try the replay and read the reports: mrsameer.github.io/tracefix

Source, Bob session logs and prompts: github.com/mrsameer/tracefix